

Freight Rate Prediction

Validation approach, data quality, and the December forecast

Machine Learning Engineer assessment · Azizul Abedin Azmi · github.com/azizulabedinazmi/freight-rate-ml

\$118.4

mean abs. error, forward-in-time CV

2.15%

median abs. % error

−15%

MAE vs. linear baseline

Summary

The task is to price 12,000 loads from November–December 2025 using 48,000 labelled loads from January–October. Because the targets lie in the *future* of the training data, I validated with expanding-window, forward-in-time folds rather than a random split. Cleaning is light (sign-flipped weights, missing values) and the model is a gradient-boosted tree trained with an absolute-error loss on $\log(\text{rate})$, which ignores a 1.4% share of corrupted labels. A small correction carries the most recent price level forward. Across three forward folds the final model cuts MAE by 15% and median percentage error by 37% relative to a robust linear baseline.

1 Problem and data

Each load has a lane (pickup, delivery, coordinates, distance), equipment type, weight, a date, and two market signals (`market_index`, `quote_signal`). The label is `posted_rate` in dollars. The assessment has three characteristics that drive every design choice below.

- **The test set is the future.** Training covers 1 Jan–31 Oct 2025; predictions are needed for 1 Nov–31 Dec 2025.
- **New cities appear.** 8 cities in the prediction set never occur in training (Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego), touching 1,447 loads.
- **The December chart has no market signals.** Its 31 fixed rows contain only lane, distance, equipment, weight and date.

2 Validation and data-split approach

2.1 Forward-in-time folds

Rates drift over the year and `market_index` moves with the date. A shuffled split would place neighbouring days of every test load in the training set and report error that will not survive contact with November. I therefore use an expanding window: each fold trains on everything before a cutoff and predicts the following two months, matching the real horizon (October → November–December).

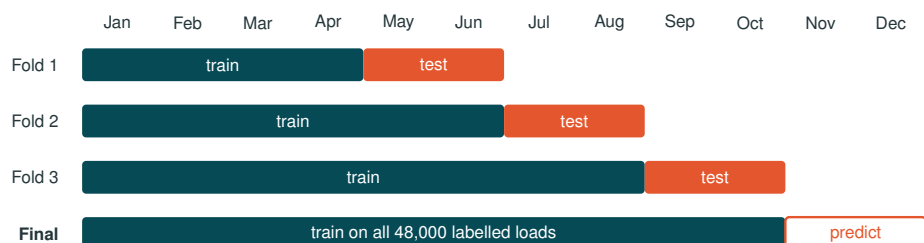


Figure 1. Expanding-window validation. The final fit uses all labelled data to predict the unseen Nov–Dec loads.

2.2 Leakage controls and metrics

- Date-level signals (daily median `market_index`, mean `quote_signal`) are computed from *feature columns only*; the label is never touched.
- Model selection and the level-offset window were chosen on the folds above; the final model is refit on all labelled rows.
- Because the grading metric is not disclosed, I report MAE, MAPE, median absolute percentage error, and RMSE in dollars.

3 What the data looks like

3.1 Price structure

Rate is multiplicative in its drivers, so everything is modelled on a log scale. A robust log-linear fit gives a distance elasticity of about 0.87, a Reefer premium of about 13% and a Flatbed premium of about 8% over Dry Van. Rate per mile falls with distance (a short-haul premium), so the relationship is not a pure power law and a non-linear model helps. Weight, market_index and quote_signal have small effects.

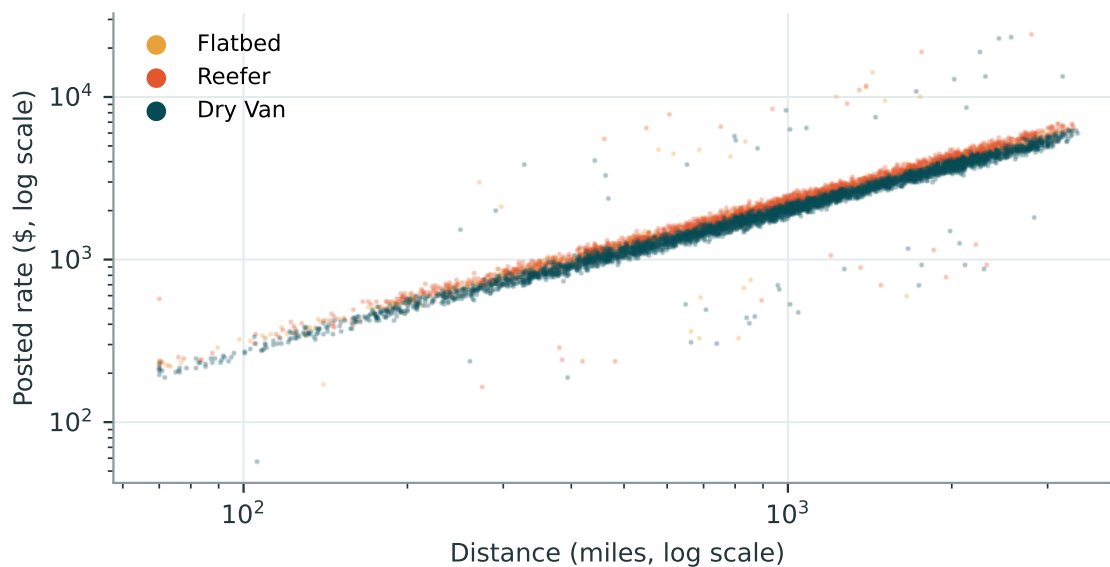


Figure 2. Rate versus distance on log axes (7,000 sampled loads). The cloud is close to linear, with equipment-specific offsets and a thin band of outliers.

3.2 The market has shifted

market_index is a date-level signal (within-day standard deviation ≈ 0.025). Its training range was 0.68–1.47 but in the prediction period it is much calmer: 0.72–1.10 with mean 0.93. Tree models cannot extrapolate beyond the training range, but the prediction period sits entirely inside it, so this is a distribution shift rather than an extrapolation problem.

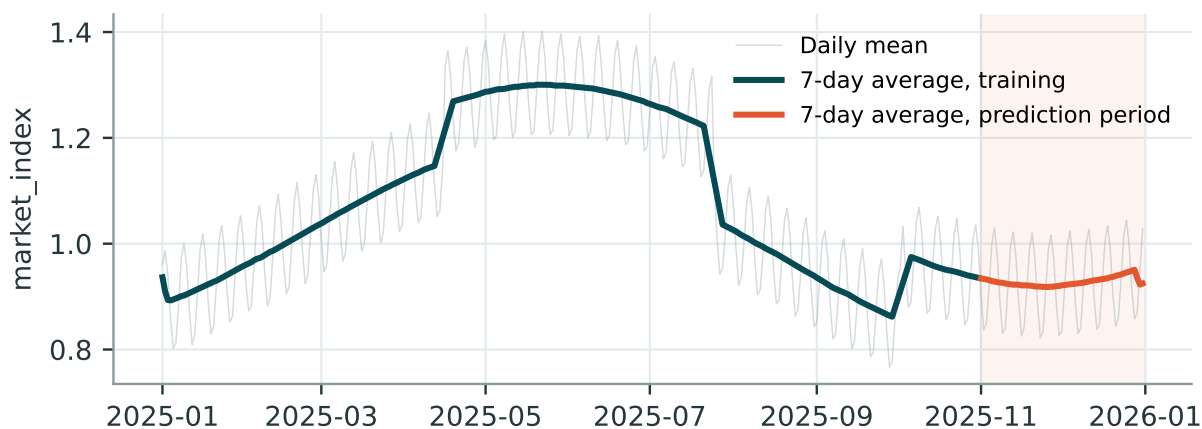


Figure 3. Daily mean market_index. It has a strong weekly cycle on top of a slow trend; the prediction period (shaded) is calmer than the spring–summer peak.

4 Data-quality issues and how they were handled

Issue	Evidence	Handling
Negative weights	292 train / 145 prediction rows; magnitudes match the positive distribution (5,000–47,500 lb)	Sign errors: use $ w $
Missing weight	300 train / 165 prediction rows	Training median
Missing <code>market_index</code>	374 train / 249 prediction rows; it is date-level	Median of that date
Corrupted labels	1.4% of rows are $\times 2.2$ – 5.4 or $\times 0.17$ – 0.45 of the expected rate, spread evenly over months, equipment and <code>quote_signal</code>	Keep rows; robust L1 loss on log rate
Unseen cities	8 cities in the prediction set absent from training	No city or coordinate features
Noisy distance	Floor of 70 miles; same lane varies by a few percent; some rows disagree with coordinates	Use the <code>distance</code> column directly

Table 1. Data-quality audit.

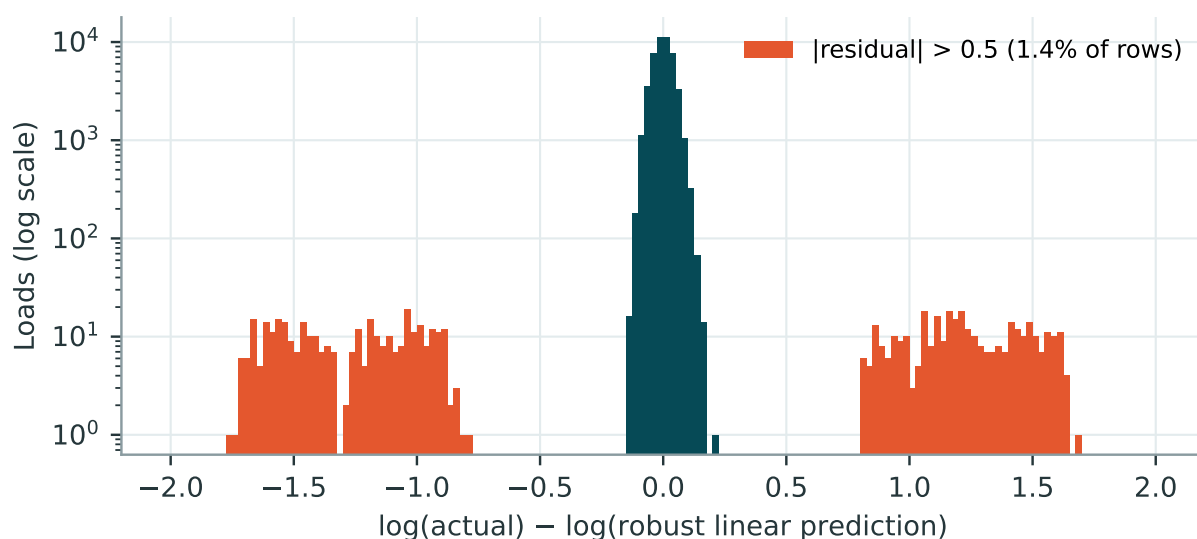


Figure 4. Residuals of a robust linear fit. The core is tight (about 4.0% noise); the coral tails are labels that no feature explains. They would dominate a squared-error objective, which is why the final model uses absolute error.

5 Modelling

5.1 Pipeline

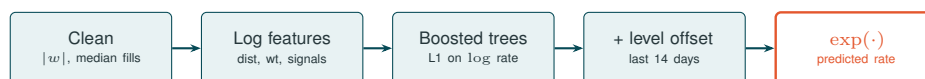


Figure 5. End-to-end pipeline implemented in `train.py`.

5.2 Features

Six inputs: log distance, log weight, log `market_index`, log `quote_signal`, and Reefer and Flatbed flags. Day-of-week, day-of-year, coordinate differences and city identity were tested in rolling validation and rejected: day-of-week and coordinates were neutral to slightly worse, and day-of-year was about 6% worse in MAE because one year of data cannot identify seasonality.

5.3 Model choice

- **Baseline:** Huber regression on log rate. Robust and interpretable, but it misses the non-linear distance curve and interactions.
- **Final:** `HistGradientBoostingRegressor` with `loss="absolute_error"` on log rate. Trees capture the short-haul premium; the L1 loss predicts a conditional median, so corrupted labels barely move it. In development, squared-error loss

was roughly 8% worse in MAE.

- **Early stopping is disabled** and the seed is fixed so results are reproducible across machines.

5.4 Recent-level correction

Residuals show a slow price-level drift (monthly median residual between about -3% and $+3\%$) that `market_index` does not explain. After fitting, I take the median residual over the last 14 days before the cutoff and add it to every prediction. For the final model this offset is $+1.2\%$. The code is short:

```
def level_offset(model, X, y, dates, cutoff):
    start = cutoff - pd.Timedelta(days=LEVEL_WINDOW_DAYS)
    recent = (dates >= start) & (dates < cutoff)
    resid = y[recent] - model.predict(X.loc[recent, FEATURES])
    return float(np.median(resid))
```

6 Results

Model (mean of 3 folds)	MAE (\$)	MAPE (%)	MedAPE (%)	RMSE (\$)
Huber log-linear (baseline)	138.9	6.03	3.42	639.9
Gradient boosting, L1 loss	125.4	5.44	2.47	634.9
Boosting + level offset (final)	118.4	5.17	2.15	632.5

Table 2. Forward-in-time validation. RMSE is dominated by corrupted labels and barely separates the models, so decisions rely on MAE and median error.

The final model beats the baseline by 15% on MAE and 37% on median percentage error. The level offset alone is worth \$7.0 of MAE.

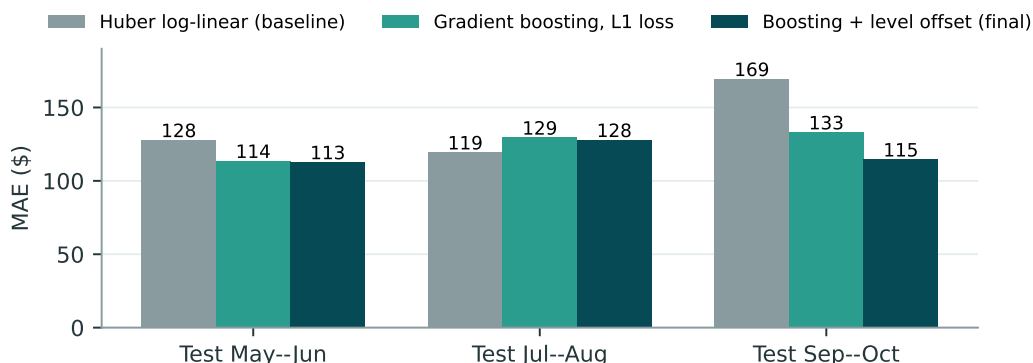
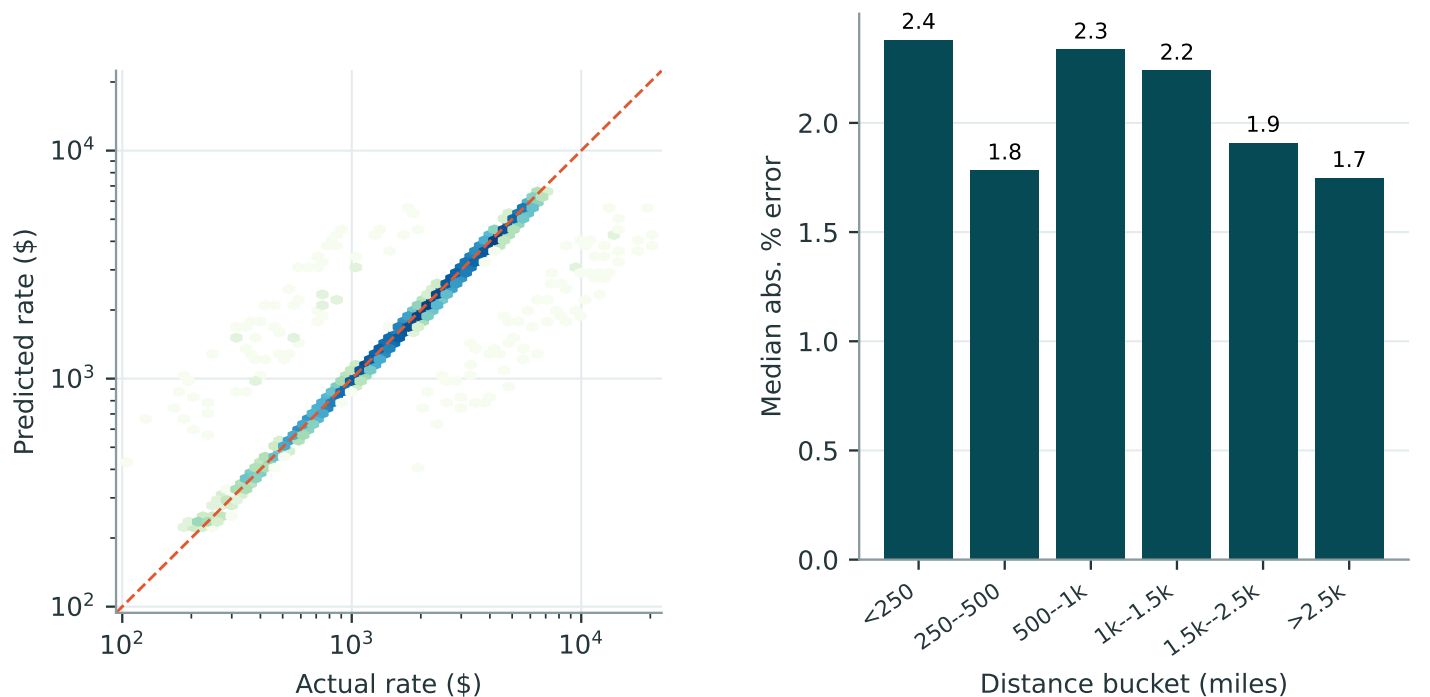


Figure 6. MAE by fold. The boosted model wins in two of three folds; in Jul–Aug a baseline fit on Jan–Jun happened to track that window slightly better.

Test window	Huber	Boosting	Final
May–Jun	127.8	113.6	112.8
Jul–Aug	119.4	129.5	127.9
Sep–Oct	169.4	133.1	114.6

Table 3. MAE (\$) per fold.

6.1 Diagnostics on the Sep–Oct fold



(a) Predicted versus actual (log axes). Points hug the diagonal; the off-diagonal arms are the corrupted labels.

(b) Median error by distance bucket: from 1.7% on the longest hauls to about 2.4% on the shortest.

Figure 7. Hold-out diagnostics. Overall median error on this fold is 2.09%.

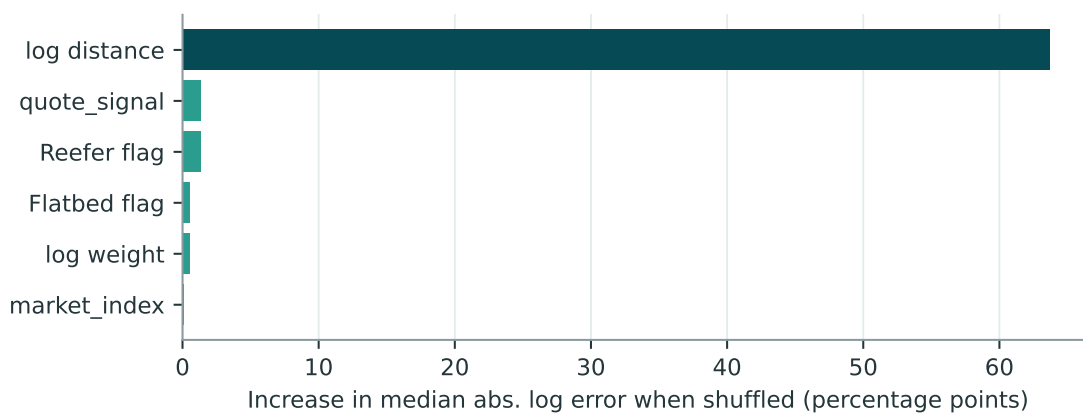


Figure 8. Permutation importance on the hold-out fold: shuffling log distance hurts far more than shuffling any other input. Market signals and equipment add small refinements, in line with the weak correlations seen in exploration.

7 Fixed December chart

The chart was produced by the provided `score.py`. Inputs are fixed (Lexington → Fort Wayne, 360 miles, Dry Van, 32,000 lb) and only the date varies. These rows carry no `market_index` or `quote_signal`, so each date uses the daily median of those signals from the December prediction loads (feature columns only).

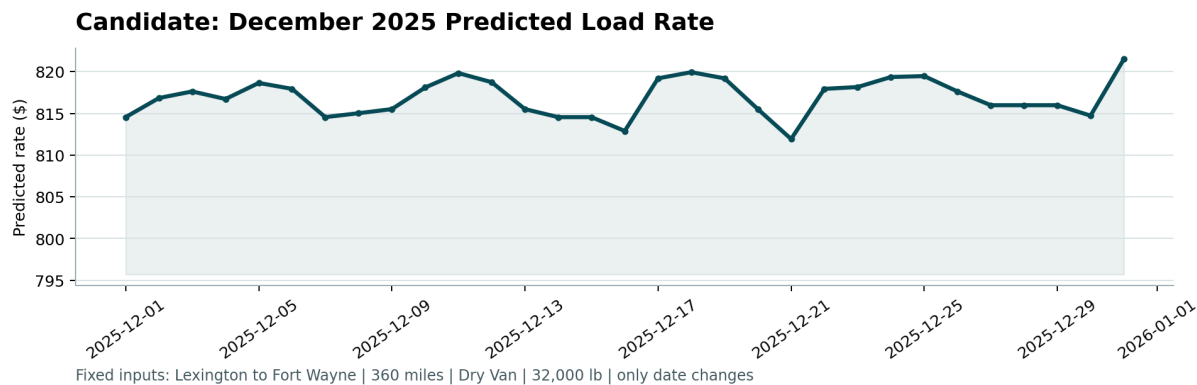


Figure 9. Predicted December 2025 rate for the fixed lane.

How to read it

Predictions range from \$812 to \$822 (mean \$817), a total spread of only 1.2%. The scorer zooms the vertical axis, which makes day-to-day moves look large. Within a month the only date-varying inputs are the two market signals, which the model finds to be weak drivers, so a flat profile is the honest forecast. I do not claim a holiday effect: the training data contains no December from which to learn one.

8 Limitations and next steps

- **Ten months of history** cannot reveal annual seasonality. With several years, month and holiday features would become learnable.
- **Level drift is carried forward, not forecast.** If November–December rates move away from the October level, the offset will not anticipate it.
- **Small tuning set.** The 14-day window and the offset benefit were selected on three folds from one year; treat the gain as moderate evidence.
- **Corrupted labels** in any hidden evaluation set will inflate RMSE for every model, including this one.
- **Next steps:** quantile models for prediction intervals, a lane-level target encoding that gracefully handles unseen cities, and an explicit detector for corrupted labels.

Reproducing the results

```
python -m pip install -r requirements.txt
python train.py
python score.py --predictions validation_predictions.csv --december-predictions data/december_chart_inputs.csv
python report/make_assets.py          # figures and numbers for this report
```